Report Advance Programming for Scientific Computing

Matteo Leone

January 29, 2026

Abstract

This project is meant to analyze the efficiency of two different solvers for the Poisson equation, this is particularly relevant both for the generic importance of this equation in scientific field and also because this equation is of particular relevance in the computational fluid dynamics field; where it arises due to the splitting operator usually applied to the Navier-Stokes equations. In this project the objective is to develop a numerical *trivially* repeatable comparison of Poisson solvers in the context of direct numerical simulations (DNS) of an incompressible fluid flow. To do so a simple NSSolver class was built to provide a test benchmark for the solvers and show a possible implementation strategy. Is important to highlight how this type of solver needs to be extremely efficient given the heavy work. In those solvers usually an explicit time integration method is employed, leaving the only equation to solve to be the one for the pressure update, here comes the relevance and importance of comparing different approaches.

In this case I confronted Multigrid solver and an FFT based solver to assess the performance difference, bottlenecks and scaling.

But first things first, I hereby present the mathematical framework and numerical discretization employed, then there will be a detailed implementation description and finally the performance comparison with why and when one is better than the other.

Contents

1 Physical and Mathematical Framework

1.1 Equation of Motion

To obtain the Equations of Motion (EOM) for an incompressible flow, the starting point is Newton's Second Law:

$$\mathbf{F} = \frac{D(m\mathbf{U})}{Dt}$$

It is possible to write the EOM for a control volume of fluid and split the left-hand side into volume forces and surface forces, $\mathbf{F}dV = \mathbf{f}_v + \mathbf{f}_s + \mathbf{f}_{ext}$. For a Newtonian, non-reacting, and non-magnetic fluid, the volume forces are written as $\mathbf{f}_v = \rho\mathbf{g}$, and the surface forces as $\mathbf{f}_s = \mu\nabla^2\mathbf{U} - \nabla p$. The total derivative on the right-hand side is rewritten in the Eulerian frame of reference employing the Reynolds Transport Theorem:

$$\frac{\partial \rho\mathbf{U}}{\partial t} + \nabla \cdot (\rho\mathbf{U}\mathbf{U})$$

The mass conservation principle, $\frac{Dm}{Dt} = 0$, is similarly expressed for a single volume of fluid as

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho\mathbf{U}) = 0$$

The hypothesis of incompressible flow implies that ρ is constant; thus, mass conservation simplifies to $\nabla \cdot \mathbf{U} = 0$, implying that the flow field is solenoidal. Combining these yields the Navier-Stokes equations for incompressible flows:

$$\begin{cases} \nabla \cdot \mathbf{U} = 0 \\ \frac{\partial \rho\mathbf{U}}{\partial t} + \mathbf{U} \cdot \nabla(\rho\mathbf{U}) = \rho\mathbf{g} + \mu\nabla^2\mathbf{U} - \nabla p + \mathbf{f}_{ext} \end{cases}$$

Since ρ is constant and non-null, it is possible to divide by it. Introducing the kinematic viscosity $\nu = \mu/\rho$, and expressing the gravitational acceleration as the gradient of a potential ($\mathbf{g} = \nabla\Psi$), the reduced pressure can be defined as $\tilde{p} = \frac{p}{\rho} - \Psi$. Defining the forcing term as $\mathbf{f} = \tilde{\mathbf{f}}_{ext}/\rho$, the EOM finally becomes:

$$\begin{cases} \nabla \cdot \mathbf{U} = 0 \\ \frac{\partial \mathbf{U}}{\partial t} + \mathbf{U} \cdot \nabla\mathbf{U} = \nu\nabla^2\mathbf{U} - \nabla\tilde{p} + \mathbf{f} \end{cases}$$

By means of dimensional analysis, we define the characteristic length L and velocity U as reference values. We introduce the following dimensionless variables, denoted by a tilde:

$$\tilde{\mathbf{x}} = \frac{\mathbf{x}}{L}, \quad \tilde{\mathbf{u}} = \frac{\mathbf{U}}{U}, \quad \tilde{t} = \frac{tU}{L}, \quad \tilde{p} = \frac{p}{\rho U^2}, \quad \tilde{\mathbf{f}} = \mathbf{f}\frac{L}{U^2} \tag{1}$$

Crucially, the time is scaled using the advective time scale L/U, which is appropriate for convection-dominated flows. Substituting these into the governing equations and simplifying yields the dimensionless Navier-Stokes equations, where the Reynolds number $Re = UL/\nu$ appears as the inverse coefficient of the viscous term:

$$\begin{cases} \tilde{\nabla} \cdot \tilde{\mathbf{u}} = 0 \\ \frac{\partial \tilde{\mathbf{u}}}{\partial \tilde{t}} + \tilde{\mathbf{u}} \cdot \tilde{\nabla}\tilde{\mathbf{u}} = -\tilde{\nabla}\tilde{p} + \frac{1}{Re}\tilde{\nabla}^2\tilde{\mathbf{u}} + \tilde{\mathbf{f}} \end{cases} \qquad \text{(Dimensionless Navier-Stokes)}$$

From this point, the necessity of HPC becomes evident. The Reynolds number Re is the critical parameter; as the problem approaches real engineering applications, Re increases drastically, becoming the leading order term. According to Kolmogorov's scale analysis for homogeneous isotropic turbulence, the smallest relevant scale is estimated as $\eta \sim LRe^{-3/4}$. To perform Direct Numerical Simulation (DNS), the discretization parameter h must resolve this scale (η), which increases the computational cost immensely, resulting in a fine mesh. It is important to highlight that the Euler equations are not the limit for $Re \to \infty$, as this limit is singular due to the change in the order of the differential equation.

From now on, the tilde notation $\tilde{(\cdot)}$ will be dropped for readability, and all variables are assumed dimensionless.

1.2 Splitting of the EOM and Numerical Discretization

Leveraging the Chorin-Temam method, it is possible to split the equations into two steps (Predictor-Corrector):

$$\text{I.} \quad \frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} + \mathbf{u}^n \cdot \nabla \mathbf{u}^n - \frac{1}{Re}\nabla^2 \mathbf{u}^n + \nabla p^n = \mathbf{f}$$

$$\text{II.} \quad \begin{cases} \frac{\mathbf{u}^{n+1} - \mathbf{u}^*}{\Delta t} + \nabla p^{n+1} - \nabla p^n = \mathbf{0} \\ \nabla \cdot \mathbf{u}^{n+1} = 0 \end{cases}$$

Correction terms (highlighted in blue) are included to ensure the overall solver maintains second-order accuracy in time. This splitting constitutes the leading-order error and serves as the reference benchmark for all subsequent discretizations. Applying the divergence operator to the second step and enforcing the solenoidal constraint on $\mathbf{u}^{n+1}$ yields the final form:

$$\begin{aligned} &\text{I.} \quad \frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} + \mathbf{u}^n \cdot \nabla \mathbf{u}^n - \frac{1}{Re}\nabla^2 \mathbf{u}^n + \nabla p^n = \mathbf{0} \\ &\text{II.} \quad \begin{cases} \nabla^2 (p^{n+1} - p^n) = \frac{\nabla \cdot \mathbf{u}^*}{\Delta t} \\ \mathbf{u}^{n+1} = \mathbf{u}^* - \nabla (p^{n+1} - p^n) \Delta t \end{cases} \end{aligned} \tag{2Step}$$

The application of the divergence operator necessitates additional boundary conditions for pressure. Specifically, applying the no-slip condition along the boundary yields:

$$\nabla p = \frac{1}{Re} \frac{\partial^2 \mathbf{u}}{\partial \mathbf{n}^2} \bigg|_{\partial \Omega}$$

A more detailed analysis (see Pope, p. 439) decomposes the pressure into particular and harmonic components, $p = p^p + p^h$. It is established that if the correct boundary conditions are applied to the harmonic solution, it remains significant only in the immediate vicinity of the wall. Consequently, the analysis is restricted to the particular solution p^p imposed with homogeneous Neumann conditions ($\partial p / \partial \mathbf{n}|_{Wall} = \mathbf{0}$). This simplification facilitates the use of the Discrete Cosine Transform (DCT) in the FFT solver.

Regarding numerical discretization, the Finite Difference (FD) approach was selected over Finite Elements (FE) or Finite Volume (FV) to prioritize a lightweight solver with low computational overhead and high parallelizability.

Regarding the numerical discretization, several options were considered:

- **Spectral Elements:** Offer high precision and low computational cost for simple implementations but lack flexibility regarding domain discretization.
- **Finite Elements (FE):** Highly flexible for complex geometries but entail significant computational and memory overhead.
- **Finite Volume (FV):** Share similar geometric flexibility with FE but generally exhibit lower order accuracy.
- **Finite Differences (FD):** Straightforward to implement and computationally efficient. The accuracy can be tuned by extending the polynomial approximation order. The primary drawback is the limitation to simple domains (e.g., boxes, spheres, cylinders), as complex geometries require advanced mapping techniques.
- **Spectral Methods (SM):** Share similar pros and cons with FD but typically involve higher computational complexity (often $\mathcal{O}(N^4 \log N)$). However, they offer superior accuracy, with error decaying as $\mathcal{O}(e^{-CN})$ compared to the algebraic convergence $\mathcal{O}(\Delta x^p)$ of p-th order FD schemes.

Given the simple domain geometry and the requirement for high accuracy, the Finite Difference (FD) and Spectral Method (SM) approaches were identified as the most suitable candidates. However, prioritizing a lightweight solver with low computational overhead and high parallelizability led to the selection of the Finite Difference approach.

For the time discretization a third order low storage approach was used, particularly the scheme is said to be 2-N as it needs only 2 more buffers to store the intermediate results.

$$
\begin{aligned}
\mathbf{Y_1} &= \mathbf{u}^n \\
\mathbf{Y_2} &= \mathbf{u}^n + \frac{64}{120}\,\Delta t\,\mathbf{f}(\mathbf{Y_1}, t^n) \\
\mathbf{Y_3} &= \mathbf{u}^n + \frac{30}{120}\,\Delta t\,\mathbf{f}(\mathbf{Y_1}, t^n) + \frac{30}{120}\,\Delta t\,\mathbf{f}(\mathbf{Y_2}, t^n + \frac{64}{120}\,\Delta t) \\
\mathbf{u}^{n+1} &= \mathbf{u}^n + \frac{30}{120}\,\Delta t\,\mathbf{f}(\mathbf{Y_1}, t^n) + \frac{90}{120}\,\Delta t\,\mathbf{f}(\mathbf{Y_3}, t^n + \frac{80}{120}\,\Delta t)
\end{aligned}
$$

the whole system can be further simplified to avoid the recalling of the same variable and therefore recomputing the same value by introducing a buffer as follows:

$$
\begin{aligned}
\mathbf{Buffer} &= \mathbf{f}(\mathbf{u}^n, t^n) \\
\mathbf{Y_2} &= \mathbf{u}^n + \frac{64}{120}\,\Delta t\,\mathbf{Buffer} \\
\mathbf{Buffer} &= \mathbf{Y_2} - \frac{34}{120}\,\Delta t\,\mathbf{Buffer} \\
\mathbf{Y_3} &= \mathbf{Buffer} + \frac{50}{120}\,\Delta t\,\mathbf{f}(\mathbf{Y_2}, t^n + \frac{64}{120}\,\Delta t) \\
\mathbf{u}^{n+1} &= \mathbf{Buffer} + \frac{90}{120}\,\Delta t\,\mathbf{f}(\mathbf{Y_3}, t^n + \frac{80}{120}\,\Delta t)
\end{aligned}
$$

This enhances the performance and avoids waste of computational power, the only loser here is the person that will need to implement it, this method is *III* order accurate in time.

1.3 Domain description

In order to avoid the spurious oscillations introduced by the decoupling of even-odd elements given the finite difference description a staggered mesh is employed.

In this particular case not all the unknown variables are defined in a single point, but the vector fields are described in a different positions particularly:

- p : defined in point (i, j, k).
- $u = \mathbf{u} \cdot \mathbf{i}$: defined in (i + 0.5, j, k).
- $v = \mathbf{u} \cdot \mathbf{j}$: defined in (i, j + 0.5, k).
- $w = \mathbf{u} \cdot \mathbf{k}$: defined in (i, j, k + 0.5).

A more graphical representation is in Figure 1

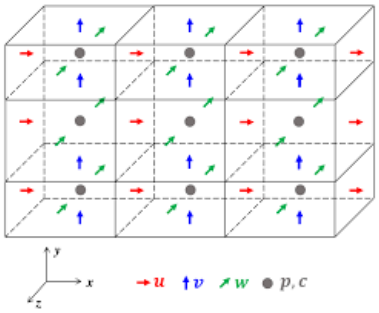

Figure 1: Representation of the staggered mesh

Is now necessary to apply some approximations as will be needed to compute the value of interested quantities in different positions from the one defined before, just to give an example: the staggered grid removes the problems of the decoupling, it can be shown better in the 2Step's last step when it's necessary to calculate the pressure gradient have:

$$\begin{cases} u^{n+1} = u^* - \Delta t \, \frac{\partial}{\partial x}(p^{n+1} - p^n) \\ v^{n+1} = v^* - \Delta t \, \frac{\partial}{\partial y}(p^{n+1} - p^n) \\ w^{n+1} = w^* - \Delta t \, \frac{\partial}{\partial z}(p^{n+1} - p^n) \end{cases}$$

And for each of the above derivatives get that:

$$\left.\frac{\partial(p^{n+1} - p^n)}{\partial x}\right|_{(i+0.5,j,k)} = \frac{[p^{n+1}(i+1,j,k) - p^n(i+1,j,k)] - [p^{n+1}(i,j,k) - p^n(i,j,k)]}{\Delta x}$$

This advantage comes to a cost, in fact there is the need to introduce an approximation, that needs to be second order in space to be consistent, for the computation of different velocities, particularly in the non linear term in the x direction can be seen: $[\mathbf{i} \cdot (\mathbf{u} \cdot \nabla(\mathbf{u})]\big|_{(i+0.5,j,k)} = (u\partial_x u + v\partial_y u + w\partial_z u)\big|_{(i+0.5,j,k)}$, but the values of v, w are not defined in the point $i + 0.5, j, k$, so a second order approximation for them is needed as well as for the derivatives so that can obtain (for simplicity and readibility is illustraded just for one term)

$$(v\partial_y u)\big|_{(i+0.5,j,k)} = \frac{v(i,j+0,5,k)+v(i,j-0.5,k)+v(i+1,j-0.5,k)+v(i+1,j+0.5,k)}{4} \, \frac{u(i+0.5,j+1,k)-u(i+0.5,j-1,k)}{2\Delta y}$$

All those logics and operators are handled in the `include/staggered_operators.hpp' file`.

1.4 The pressure solvers

The pressure solver is the computational bottleneck of the projection method. Here, the mathematical formulation of the two implemented strategies is detailed. Both solvers implementation tackle a linear system defined by the second order finite difference that can be sought in the form of $\boldsymbol{A}\ \boldsymbol{p} = \boldsymbol{f}$. The solve strategy is the main difference.

- **Fast Poisson solver:** Based on real-to-real fast Fourier transforms using `fftw3`.
- **Geometric Multigrid solver:** Based on `PETSc`, this is one of the most robust strategies for solving the Poisson problem.

1.4.1 Fast Poisson solver

This solver is based on the key idea that in the spectral space, the Poisson problem can be solved algebraically, leading to significant performance gains.

It is well known that given a pressure field $p(\mathbf{x})$ and the Poisson equation $\nabla^2 p = f(\mathbf{x})$, one can apply the Fourier Transform operator $\mathcal{F}$ to diagonalize the differential operator. In the continuous case, applying the transform yields:

$$\tilde{p}(\mathbf{k}) = \mathcal{F}[p(\mathbf{x})], \quad -|\mathbf{k}|^2 \tilde{p}(\mathbf{k}) = \tilde{f}(\mathbf{k})$$

Since the operator is linear, in 3D this simplifies to:

$$\Lambda_{ijk} \tilde{p}(\mathbf{k}) = \tilde{f}(\mathbf{k})$$

where λ represents the eigenvalues of the Laplacian operator. The solution in spectral space is obtained by simple division, and the physical solution is recovered via the inverse transform $p(\mathbf{x}) = \mathcal{F}^{-1}[\tilde{p}(\mathbf{k})] = \mathcal{F}^{-1}[\tilde{f}(\mathbf{k})/\Lambda_{ijk}]$.

The implemented algorithm builds on this theory but adapts it for the discrete domain to ensure high accuracy. It leverages `fftw3` real-to-real transforms (DCT/DST) to enhance memory efficiency. Crucially, the spectral solution uses the exact eigenvalues of the discrete finite difference operator rather than the continuous approximation $(-k^2)$.

In fact, given the standard second-order central difference stencil for the second derivative:

$$\frac{\partial^2 p}{\partial x^2} \approx \frac{p_{j-1} - 2p_j + p_{j+1}}{h^2} \tag{2}$$

build the second order finite difference laplacian matrix problem $\boldsymbol{A}\ \boldsymbol{p}= \boldsymbol{f}$ and solve the eigenvalue problem $\boldsymbol{A}\ \boldsymbol{p}= \lambda\ \boldsymbol{p}$ satisfying the recurrence relation

$$(p_{j-1} - 2p_j + p_{j+1})/h^2 = \lambda_k p_j$$

for an eigenvector $p_j := p_0 e^{i\theta_k j}$ (where i is the imaginary unit) : Using the spectral ansatz where $\theta_k = \frac{k\pi}{N-1}$, the discrete eigenvalues are derived as:

$$\lambda_k = \frac{2\cos\left(\frac{k\pi}{N-1}\right) - 2}{h^2} \tag{3}$$

for the $k = 0, \ldots, k_{n-1}$.

In the 3D solver, the total eigenvalue for a mode with indices (i, j, k) is the sum of these directional eigenvalues due to the linearity of the operators:

$$\Lambda_{ijk} = \lambda_i^{(x)} + \lambda_j^{(y)} + \lambda_k^{(z)}$$

This formulation ensures that the spectral inversion exactly matches the finite difference discretization used in the rest of the simulation, it's just a different more efficent algorithm to solve the linear system that relyies on heavier analysis; but of course nothing comes for free, and it has in fact some limitations, is necessary that the boundary conditions on the two ends are coincident. The solver is implemented only for homogeneus Neumann and Dirichlet boundary conditions, and also when parallelized in order to be efficent and limit the number of communications there is the need to transpose the data among the different diirections, to do so I leveraged the library `2Decomp` in particular the `C++` traduction from fortran, I needed to modify it partly but I didn't reinvent the wheel.

The power of the solver is in its simplicity, there are no iterations and by precomputing the eigenvalues solving the algebraic equation is even faster since less time is spent computing the sines and the inverses of the sum.

The back substitution step is done leveraging thread level parallelism as well since the algorithm is *embarrassingly parallel*. The algorithm uses `FFTW3` to perform real-to-real transforms (DCT/DST). Specifically, Homogeneous Neumann conditions utilize the Discrete Cosine Transform (DCT-I), while Homogeneous Dirichlet conditions utilize the Discrete Sine Transform (DST-I). This approach is iteration-free and computationally efficient $\mathcal{O}(N^3 \log N)$ in 3D.

1.4.2 Geometric Multigrid Poisson solver

While the Fast Poisson solver exploits the spectral properties of the operator to solve the system directly, the Geometric Multigrid (GMG) solver tackles the linear system $\mathbf{Ap} = \mathbf{f}$ in physical space using an iterative approach.

It is well documented that standard stationary iterative methods (such as Jacobi or Gauss-Seidel) effectively reduce high-frequency error components (smoothing) but stall when reducing low-frequency errors. The GMG algorithm overcomes this by constructing a hierarchy of discretized domains $\Omega_h, \Omega_{2h}, \ldots, \Omega_L$ with increasingly coarser mesh sizes. The key insight is that a low-frequency error on a fine grid appears as a high-frequency error on a coarser grid, where it can be smoothed efficiently.

The implementation leverages the `PETSc` library, specifically the `DMDA` (Distributed Mesh Data) management, to handle the grid hierarchy and parallel data exchange automatically, to encapsulate it and again make an API with a common interface I build a PETScDecomp class to handle domain decomposition.

The solver employs a standard V-cycle scheme consisting of the following steps:

1. **Pre-smoothing:** Apply ν_1 iterations of a smoother (e.g., SOR or Chebyshev) on the fine grid to dampen high-frequency errors.

2. **Residual Computation:** Compute the residual on the fine grid:

$$\mathbf{r}_h = \mathbf{f}_h - \mathbf{A}_h \mathbf{p}_h$$

3. **Restriction:** Transfer the residual to the coarser grid using a restriction operator I_h^{2h}:

$$\mathbf{r}_{2h} = I_h^{2h} \mathbf{r}_h$$

4. **Coarse Grid Correction:** Solve the error equation $\mathbf{A}_{2h}\mathbf{e}_{2h} = \mathbf{r}_{2h}$ on the coarse grid (recursively).

5. **Prolongation and Correction:** Interpolate the error correction back to the fine grid using a prolongation operator I_{2h}^{h} and update the solution:

$$\mathbf{p}_h \leftarrow \mathbf{p}_h + I_{2h}^{h}\mathbf{e}_{2h}$$

6. **Post-smoothing:** Apply ν_2 iterations of the smoother to remove any high-frequency noise introduced by the interpolation.

Unlike the FFT solver, which requires a global all-to-all communication , the GMG approach primarily involves local point-to-point communications (halo exchanges) during smoothing and inter-grid transfers. This theoretically allows for better scalability on massive clusters, albeit with a higher computational cost per degree of freedom compared to the highly optimized FFTW implementation on structured Cartesian grids.

The details of the implications on the datastructures for the solvers will be treated in the section dedicated to parallel implementation.

The implementation relies on `PETSc`'s `KSP` interface with a `PCMG` preconditioner by default. While for the **Spectral** solver there are no settings to manually adjust in the case of the **Multigrid** settings are what can make a solver efficient and scalable, the detailed setting implementation and handling will be detatiled in subsection 4.2. This abstraction allows flexibility in choosing different smoothers or bottom-level solvers without modifying the core solver, in fact even after compilation `PETSc` allows to use flags when launching the executable that can heavily modify the implementation of the preconditioner, type of cycle (V vs W), numer of coarse levels, maximum iterations, realtive tolerances and many more, leveraging this dependency is what made this part possible , and encapsulationg everithing in a class what gave the flexibilty to modify and integrate such a verbose and complex API into a simple solve().

2 Modern C++ Design and abstractions

In the development of a high performance solver, the software architecture plays a pivotal role equal to the numerical scheme itself. The design philosophy of this project leverages modern C++ features, such as templates, concepts, and policy-based design, to achieve zero-cost abstractions. To do so the standard chosen is the C++23. This ensures that the code remains modular and readable without sacrificing the runtime performance and the use of concepts allows nice error logs instead of infinite dump from the compiler.

Portability is also a key aspect to avoid *works on my machine* situations, to ensure to have nice portable code I chose to use nix flakes. Nix is a package manager, is bigger than Arch's pacman and easier to use than apt, also is able to give the latest packages keeping them stable and declarative. I chose nix over the mkmodules due to the inability to use C++23 features on them given the old version of the gcc compiler.

The project depencies are:

- PETSc to handle the `MGSolver`.
- FFTW for the `FastPoissonSolver`.
- tbb for the thread level parallelization.
- MPI and OpenMP to hanlde the parallelization over processes and threads.
- Python basic packages like sympy to write the manufactured solutions for tests, numpy for solving simple linear system and black for indenting in a consistent way.

Another note, I chose to use templated classes both for generality and performance (also since the codebase is small the usual critiques as long compilation times and big executables are out of scope), there are some strict checks using the `requires` keyword to ensure that the templates are not source of confusion for the user.

All the implemented class a part for the *Decomposers* are in the numPDE namespace.

2.1 Data Structures for Scientific Computing

At the core of the solver lies the management of the discretized field data. To handle the complexity of the staggered grid arrangement, where velocity components and pressure are defined at different spatial locations I implemented a special `Tensor` class that is able to handle multidimensional data, the option was to use `std::md_span`, one of the nicest proposals of C++23, however as of the today the compiler support is very limited, not even gcc 15.2.0, and neither clang 19.1.7 support them (compiler support page), the alternative would have been to use the kokkos implementation, however the inability to use proxy elements and the desire to experiment with expression templates and build something mine lead to this choice.

Together with the `Tensor` class there are also `numPDE::Vector`, `numPDE::Array` and `numPDE::ProxyElement` in order to be able to have nice support for expression templates all the datastructures are defined in the `include/datastructs` directory.

The class is defined in the tensors.hpp file.

```
enum TypeIndex
{
    // HPC-like  indexing like T(i, j, k) => datas[ i + y*Nx + k*Nx*Ny ]
```

```
    ROW_MAJOR,
    // CUDA-like indexing like T(i, j, k) => datas[ i*Nx*Ny + j*Ny + k ]
    COMPACT
};
template <typename T, size_t RANK = DEF_DIM,
size_t N_DIMS = RANK, TypeIndex TYPE = ROW_MAJOR>
struct Tensor : public Expr<Tensor<T, RANK, N_DIMS, TYPE>>
```

The class supports different type of indexing, ROW_MAJOR is the default one since is very typical for HPC applications and general storing of multidimensional data.

This class is purely a container, with a nice twist, since I could act on the `operator=` I chose to use a nice trick seen in the book `C++ high performance`, so that the behavior is different if the data to handle are scalarlike or vectorlike and all at compile time meaning is possible to do this for example:

```
auto C   = h_U(i, j, k);     // center
auto E   = h_U(i + 1, j, k); // east
auto W   = h_U(i - 1, j, k); // west
auto N   = h_U(i, j + 1, k); // north
auto S   = h_U(i, j - 1, k); // south
auto Top = h_U(i, j, k + 1); // top
auto B   = h_U(i, j, k - 1); // bottom

return  ris numPDE::Array<h_U::type_value, h_U::N_DIMS>
{(E + W + N + S + Top + B - 6.0 * C) / (h*h)};
```

the accessed `Tensor`'s element simply returns a `ProxyElement` that consists in a pointer and a size relying on another grate feature of `C++20` the `std::span` class.

The readability gain makes the code much easier to write allows to avoid useless loop and is easier to maintain also this comes at nearly 0 cost in fact the index calculation is done once (instead of 3 times to access the 3 element of the velocity tensor) and returns simply 2 std::size_t instead of 3 Tensor::type_value (which would mean 3 doubles in my specific tests implementation).

The development of this class made so much more enjoyable the development of the staggered operators needed for the `NSSolver` class.

The `Array` class is meant to handle the algebric operations on Tensor's elements more readable and expression-templetize them. In fact by leveraging CRTP (i.e. compile time polymorphism) and operator overloading my class is able to perform simple operations like sum or elementwise product in a readable and efficient way, the assignment is also made super easy thanks to the already mentioned `ProxyElement` class, so that is possible to do this as well.

```
// Returns a numPDE::Tensor<double, 4, 3, TypeIndex::ROW_MAJOR> element.
auto U = numPDE::make_vector_field<double, 3>({3, 3, 2});
for( const auto [k, j, i] : U.int_elems())
U(i, j, k) = {1.0, 0.0, 0.0};
```

Note the `int_elems()` method, which returns a `std::ranges::views::cartesian_product` object. This facilitates iteration over the internal elements of the Tensor in a consistent manner typical of PDE solvers. (An accessor for the entire domain, `all_elems()`, is also available).

To leverage thread-level parallelism effectively, the `tbb::parallel_for` function was selected. One might question this choice over the cleaner, standard algorithms templated on `std::execution_policy`. The decision stems from the extensive use of `std::views` for lazy evaluation; unfortunately, standard algorithms currently lack robust support for parallelizing views.

However, standard algorithms with specific `std::execution_policy` were still employed whenever possible (and effectively parallelizable), as they offer superior abstraction and often optimized implementations for standard tasks.

Given that the solver algorithm is fully explicit, no mutex or guard mechanisms are required.

To mitigate the verbose syntax of `tbb::parallel_for`, custom templated wrapper functions were implemented within the `trd_par` namespace (see `include/parallel_for_loops.hpp`).

With these helpers, the resulting code structure is as follows:

```
auto instruction = [&](auto i, auto j, auto k)
{
    const auto pos   = get_pos(i, j, k);
    const auto f_V   = predictor_f(m_V, i, j, k, r_inps.constants);
    const auto f_ext = r_inps.v_BC.f(pos);
    Buff(i, j, k)    = f_V + f_ext;
};
trd_par::parallel_for_int_elems(m_P.get_sizes(), instruction);
```

Pretty explicit and clear but most importantly efficient!

2.2 Cache Locality

To maximize performance on modern CPU architectures, memory access patterns must respect the cache hierarchy. As already specified I adopted a **row-major** memory layout.

This layout is critical for the performance of inner loops. By iterating elements in row major and consistent order, is possible to maximize spatial locality and enable the CPU's hardware prefetcher to efficiently load cache lines, significantly reducing cache misses during stencil operations. Furthermore, during MPI communications, data is packed into contiguous buffers before transmission to minimize latency and memory fragmentation, in fact only in specific cases the asynchronous data transmission of smaller buffers is more efficient than providing a copy, transmit the data and copy them in the final destination due to both cache hierarchy and per message overheads.

The whole tensor class is thought to leverage this concept and make it as hidden as possible to the end user, there are checks on the index to avoid reading or writing on non-owned memory that can be disabled at compile time.

In the performance chapter is shown using GNU/perf analyzer the nice cache predictability pattern possible thanks to the consistent implementation.

2.3 The Power of Abstraction

A major goal of the architecture was to decouple the "physics" of the Navier-Stokes equations from the "numerics" of time integration and the "topology" of parallel decomposition. The decoupling also allows to divide the task and build a class to each task, this way the code is easier to test, verify and maintain.

Almost all classes have unit tests to ensure convergence, correct functionality and that can also act as tutorials and self documentation.

This separation is evident in the `NSSolver` class. It defines the single instructions for each step but delegates the temporal advancement to a separate `RKStepper` class and the domain management to a `Decomp` object, the solutions are stored into a `numPDE::Tensor` object. This separation of concerns allows to inspect, test and optimize each component in isolation. For instance, the time-stepping logic is agnostic to the underlying physics, requiring only that the system satisfies a specific Concept (`SolverConc`). In fact this class is tested in the specific `tests/parallel/mock_timestepper.cpp`, where the third order convergence is tested on a non linear ODE.

In this case the readability is a bit lost since the time integration algorithm is not the most straightforward, however this mock class can be seen as a nice self documentation tutorial candidate.

2.4 Policy-based Design

To support multiple solver strategies without the runtime cost of virtual functions (dynamic polymorphism), I employed **Policy-based Design**. This technique uses `C++` templates to assemble classes with specific behaviors at compile-time.

As seen in the `NSSolver` definition:

```
template <SolvePolicy solveP, DecomposeConc Decomp>
struct NSSolver { ... }
```

Here, `solveP` is a policy parameters described by a simple `enum class`.

- **Solver Policy (`solveP`):** Allows the user to switch between different pressure solvers (namely `SolvePolicy::Fourier` vs. `SolvePolicy::MultiGrid`) at compile time. Since the compiler generates a specialized class for each policy, calls to the solver are statically bound and can be fully inlined, eliminating v-table lookup overhead that even if most of the times are negligible and not the algorithm bottleneck are also avoidable.

- **Decomposition Policy (`Decomp`):** Abstracts the parallel decomposer class (`PETScDecomp` vs. `NewDecomp`). This allows the code to run on different decomposer simply by changing a template argument.

The beauty of concepts is that one can describe what the template parameter is expected to do so that if this promise is broken compilation is stopped limiting and preventing early bugs that would otherwise turn into a nightmare (Given also the complexity of debugging in multi-process environment).

This policy based design allow to choose the correct templated specialization of the pressure solver.

```
template <SolvePolicy solveP, DecomposeConc Decomp>
struct PressureSolver;
```

Leveraging the single responsibility principle I preferred whenever possible to opt for composition over inheritance.

```
template <SolvePolicy solveP, DecomposeConc Decomp>
struct NSSolver
{
    using T          = Decomp::value_type;
    using type_solve = Tensor<T, 4, 3, TypeIndex::ROW_MAJOR>;
    using value_type = T;

    Decomp&                          r_dec;
    PressureSolver<solveP, Decomp> pSolve;
    NS_input<T>&                     r_inps;
    // Latest timestep solution tensors
    type_solve                                m_V;
    Tensor<T, 3, 3, TypeIndex::ROW_MAJOR> m_P;
    RKStepper<type_solve>                     stepper;
    bool                                      m_verbose = false;
```

Classes hierarchy: Each `PressureSolver<Policy, Decomp>` inherit from the specific solver it needs to perform the pressure correction step and provides a common interface to perform the pressure correction step. I chose to let the `PressureSolver` perform the last velocity update since it can leverage data locality and avoid copies and unnecessary data movements.

One may argue that this class specialization could have been avoided using a specialized `PoissonSolverConc` concept and making the `NSSolver` templatized over it, I chose to avoid it because the pressure solver does much more than this and is in fact responsible to perform a whole step and encapsulating this complex logic into a set of functions would have been quite difficult and would have killed the modularity of the project breaking the single responsibility principle I tried to enforce into the whole codebase.

2.5 Portability and Reproducibility

Ensuring the reproducibility of results is a primary objective of this project. To achieve a hermetic and deterministic build environment, I employed **Nix Flakes**. Unlike traditional package managers or containers (e.g., Dockerfiles) which can suffer from temporal drift, Nix Flakes utilizes a purely declarative configuration model. This guarantees that every dependency from the compiler toolchain to the MPI implementation is pinned to a specific commit hash of the `nixpkgs` repository.

The configuration, defined in `flake.nix`, explicitly manages the entire dependency graph. This allows us to leverage cutting-edge tools, **GCC 15**, **Clang-Tools** alongside complex scientific libraries (**PETSc**, **FFTW3**, **OpenMPI** and **tbb** to name a few) without polluting the host system.

Crucially, the environment enforces purity as seen in Listing 1, the `shellHook` automatically sanitizes the runtime environment by unsetting `LD_LIBRARY_PATH`, preventing accidental linking against system-wide libraries which often leads to ”works on my

machine" discrepancies. Furthermore, it automates the setup of critical environment variables (e.g., `PETSC_DIR`, `EIGEN_INCLUDE_DIR`), eliminating the need for manual configuration scripts.

Listing 1: Snippet of the Nix Flake configuration file.

```
devShells.default = pkgs.mkShell {
    buildInputs = [
    # Compilers & MPI
    pkgs.gcc15
    pkgs.openmpi
    # Scientific Libraries
    pkgs.fftw
    pkgs.petsc
    pkgs.eigen
    pkgs.tbb
    ];

    shellHook = ''
    # Automate environment setup
    export PETSC_DIR="${pkgs.petsc}"
    export FFTW_INCLUDE_DIR="${pkgs.fftw}/include"

    # Enforce purity by ignoring host libraries
    unset LD_LIBRARY_PATH
    echo "PACS environment activated"
    '';
};
```

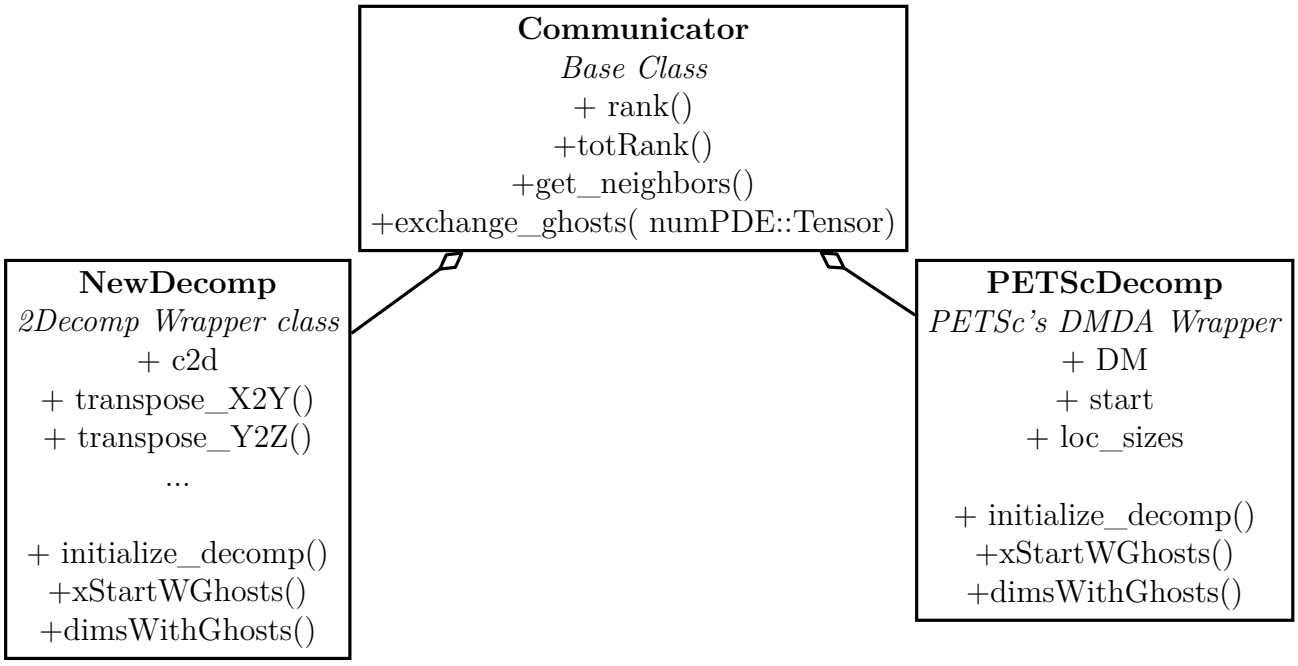

Figure 2: Scheme of the decomposers

3 Software Architecture

The main numerical investigation objective for this project is the pressure solver efficiency, however in order to test it I built a full NSSolver class, in this chapter I'll explain how I designed the code.

In general I wanted each class to have strictly the needed members avoiding as much as possible double copies in fact I used (heavily) references, also I wanted each class to be able to store and solve the solution and limit memory footprint, since inheritance can be quite a tricky aspect I tried to limit it as much as possible unless it was the most natural choice.

Going step by step:

3.1 Decomposers

The Decomposers are the classes that handle the domain decomposition logics and in general the MPI communication, initializations and more.

To do so I built the `Communicator` class, that handles MPI initialization and finalization consistently avoiding the risk of double initialization or missing finalization and in general MPI related errors.

As presented in Figure 2, from communicator 2 children classes arise `NewDecomp` and `PETScDecomp`, both are templated class that handle domain decomposition logics.

3.2 Pressure Solver

As specified above I chose to use a Policy pattern for the pressure solver. Each pressure solver has a specialization and is expected to be able to handle the pressure correction step:

- Solve the pressure equation: $\Delta\Phi = \nabla \cdot \mathbf{V}^*/dt$
- Update Pressure and velocity: $\mathbf{V}^{n+1} = \mathbf{V}^* - dt\nabla\Phi \qquad \& \qquad P^{n+1} = P^n + \Phi$

The `PressureSolver` struct is an empty templated forward declaration, then each specialization needs to be defined. (See `include/pressure_solver.hpp` for more details) I defined 3 specializations:

- `numPDE::SolvePolicy::MultiGrid` that leverages the `MultigridPoissonSolver` class.
- `numPDE::SolvePolicy::Fourier` that leverages the `FastPoissonSolver` class.
- `numPDE::SolvePolicy::None` for debug purposes of the `NSSolver` class.

The `PressureSolver` has a `correct_pressure()` method that takes as inputs the velocity field $\mathbf{V}^*$ and the pressure field by reference, and performs the needed updates.

Here each solver leverages its internal data structure to perform the pressure correction step in the most efficient way and limiting the copies needed.

As visible in Figure 3 MultiGrid and Fourier implementation have respectively `extrapolate_div_on_side()` and `compute_div_on_sides()`; this functions employ a second order approximation (using a third order polynomial as detailed in subsubsection 4.2.2) to compute the divergence field on the sides of the domain, this step is needed for the Fourier specialization since the employed FFT needs the data to be defined also on the domain's boundary, therefore the values need to be computed in order to *feed* the solver an appropriate problem, on the other hand the MultiGrid solver does not need the value of the divergence on the boundary since the said interpolation is forced in the matrix definition, however the value on the boundary is needed for the velocity update step, therefore the same interpolation is employed.

The solve call is delegated to the needed backend that is in this case obtained by means of inheritance since the flow of information it's straightforward.

3.3 The NSSolver class

The `NSSolver` builds on the `PressureSolver` class. Figure 4 shows composition over inheritance in this case (as implies the full diamond), the needed inputs and Decomp instance are taken by reference, since they are *used*, not owned by the solver itself. The solver also owns a stepper instance, an helper class that handles the buffer needed for the Runge-Kutta time stepping scheme, the templated solver expects some attributes and methods, all of which are defined in the `SolverConc` again, if one is missing compilation fails and bugs are caught early.

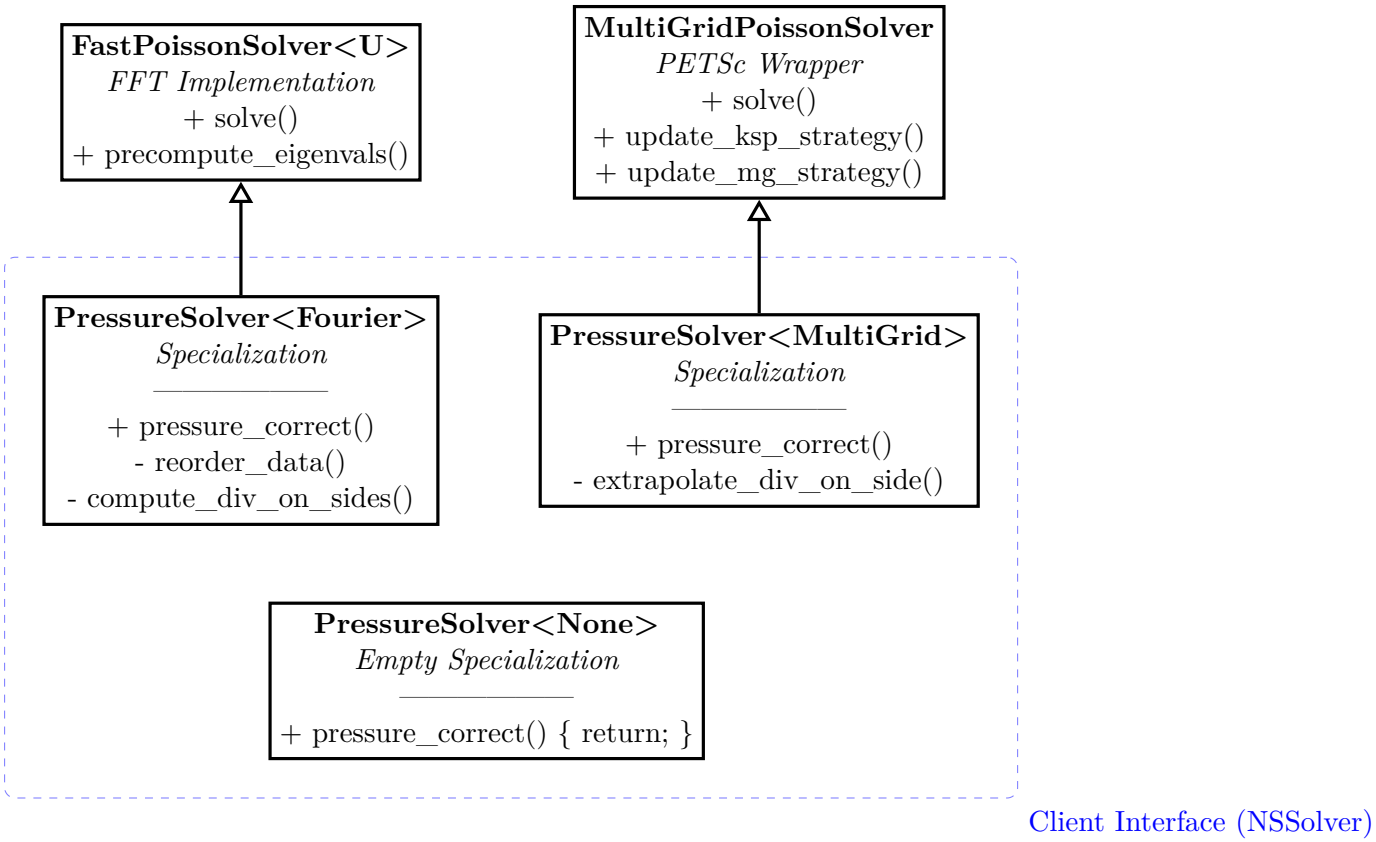

Figure 3: **Compile-Time Selection** The class `PressureSolver` inherits from different backends based on `SolvePolicy` template enum.

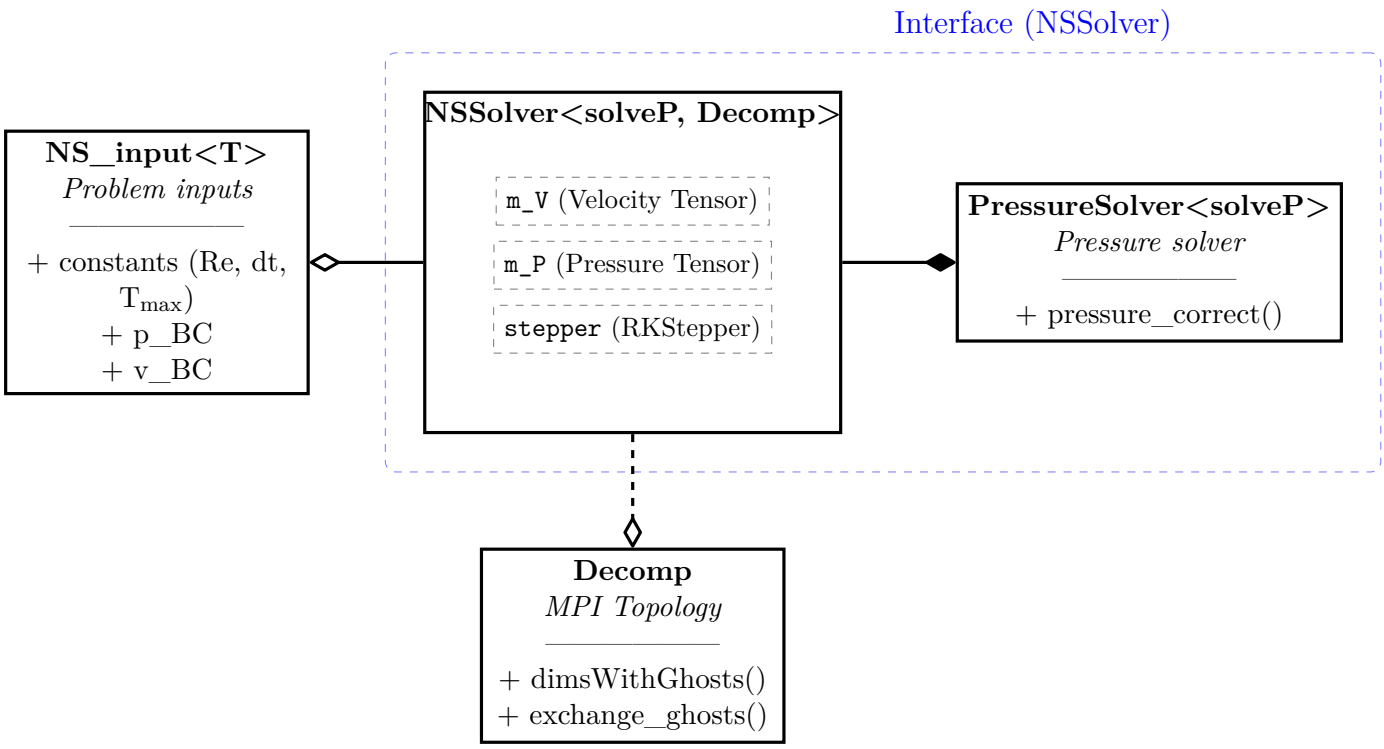

Figure 4: UML diagram of the NSSolver class scheme

3.4 Parallelization

To handle parallelization a `Communicator` class was built that handles MPI initialization and finalization, to handle special cases it's possible to `release_mpi_ownership()`.

The domain is decomposed in pencils accordingly to the row major order of the Tensor's class.

This class has also the methods to allow tensors to exchange boundaries nodes, this operation is done by copying all the elements of the side faces into a buffer that is then sent to the neighbours process, this was favoured instead of asynchronous single rows communication because allows to have more scalable results thanks to fewer communications and a more cache line friendlyness.

From the `Communicator` there are 2 children classes `NewDecomp` that is born to encapsulate and wrap the verbose functionalities of the `C2Decomp` library, and with the same idea in mind there is `PETScDecomp` class that is meant to handle the PETSc environment and the relative PETSc DMDA.

To handle thread level parallelization I employed industry standard libraries as `OpenMP` and `tbb`.

I didn't specialize any instructions for `OpenMP` since it's used only in the `apply_bc()` loop, while I instead made some more experiments with `tbb`. I simply encapsulated its verbose sintax in a templated function, that takes an `std::array` and builds from that making one more domain pencil decomposition. The advantages are that tbb is automatically able to know how many threads each process has available and assigns them the work to do, also the decomposition is done automatically at zero cost for the user and finally there is no need for complex `std::mutex` logics.

4 Pressure Solvers Parallel Implementation

Having established the mathematical framework, I now address the parallel architecture of the solvers. The primary challenge in distributed memory environments is minimizing communication overhead. For all solvers, domain decomposition and MPI handling are abstracted via the `Decomposers` class strategies.

4.1 Fast Poisson Solver

The efficiency of the Fast Poisson solver relies on performing three specific operations with minimal latency:

- **Global Transposition:** Realigning data in memory so that the dimension being transformed is contiguous (row-major order).
- **Spectral Transforms:** Computing forward and inverse Fast Fourier Transforms.
- **Spectral Decay:** Solving the algebraic relation $\hat{p} = \hat{f}/\lambda$ in the frequency domain.

4.1.1 The Transform Engine

The `FFTW3` library was selected for its industry-standard performance and portability. I utilize the `r2r` (real-to-real) interface to exploit physical symmetries, effectively doubling performance compared to complex-to-complex transforms:

- **Homogeneous Dirichlet:** Uses `FFTW_RODFT00` (DST-I), which naturally enforces zero values at boundaries via odd symmetry.
- **Homogeneous Neumann:** Uses `FFTW_REDFT00` (DCT-I), which enforces zero derivatives at boundaries via even symmetry.

4.1.2 Domain Decomposition & Pencil Transposition

To enable 3D transforms, I utilize the `C2Decomp` library, a modern `C++` port of the highly scalable Fortran `2Decomp`. Standard 1D slab decomposition limits parallelism ($P < N$). Instead, I employ a **2D Pencil Decomposition** (see Figure 5).

This strategy permits massively parallel scaling ($P < N^2$) and ensures that for any given direction (x, y, or z), the solver can locally access the entire line of data required by the FFT backend. The transposition is performed asynchronously using non-blocking MPI calls. Note that this operation cannot be performed in-place; distinct auxiliary buffers are required for the transposed states leading to higher memory footprint of the solver. This causes the `FastPoissonSolver` class to need 2 extra `std::vector`s to handle the transposed data.

4.1.3 Spectral Solution & Nullspace Handling

The solution step is fully explicit. The eigenvalues Λ_{ijk} are precomputed during initialization, reducing the solver to a vectorized product. To leverage modern `C++` threading, I utilize standard algorithms with parallel execution policies:

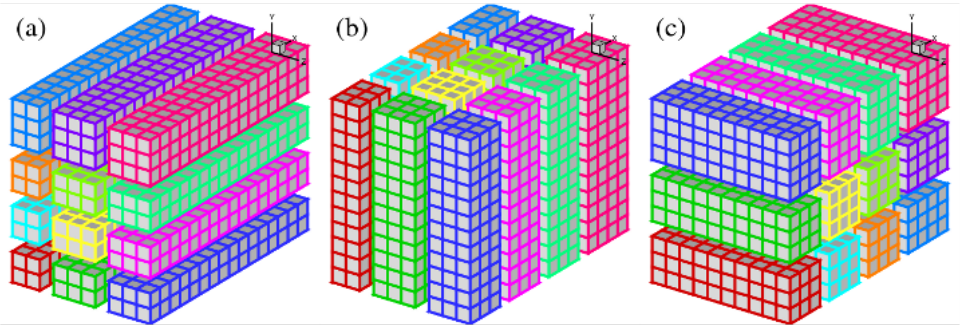

Figure 5: Schematic of the 2D Pencil Decomposition. Data ownership is rotated to ensure contiguous memory access during X, Y, and Z transforms, maximizing cache locality.

```
template <typename T>
void FastPoissonSolver<T>::solve_spectral()
{
    // Parallel element-wise multiplication
    std::transform(std::execution::par_unseq,
                   m_data3.begin(), m_data3.end(),
                   eigenvals.begin(), m_data3.begin(),
                   [](T d, T eig) -> T { return d * eig; });
}
```

Singularity Handling: The Poisson equation with pure Neumann boundaries is singular (defined only up to a constant). In the spectral domain, this manifests as a zero eigenvalue at the fundamental mode ($k = 0$), leading to a potential division by zero. I address this by explicitly setting the inverse eigenvalue to zero for the $(0, 0, 0)$ mode. This effectively regularizes the solution by setting the mean pressure to zero, preventing numerical drift.

Numerical help for the solver: Given the fftw's library backend is relevant to notice, in order to have a fftw3-friendly discretization, that the documentation specifies: 'the standard FFTW distribution works most efficiently for arrays whose size can be factored into small primes (2, 3, 5, and 7), and otherwise it uses a slower general-purpose routine.'

4.2 Geometric Multigrid Solver

Unlike the FFT solver, which relies on global spectral operations, the Geometric Multigrid (GMG) solver operates in physical space. This necessitates a distinct parallel strategy handled through the `PETSc` library.

4.2.1 Architecture and Data Structures

The integration challenges stem from the impedance mismatch between my custom `Tensor` class (optimized for stencil operations) and PETSc's internal vector layout.

- **Global vs. Local Indexing:** PETSc manages the distributed solution vector using a global indexing scheme. I implemented a lightweight translation layer in the `PETScDecomp` class that maps my local (i, j, k) pencil coordinates to PETSc's global indices during the assembly of the right-hand side (RHS) vector.

- **Ghost Exchange:** While my solvers manually handle halo exchanges, the linear system solve delegates this entirely to PETSc's `DMDA` (Distributed Mesh) object. This encapsulates the communication complexity, ensuring that matrix-vector products within the Krylov subspace methods (KSP) automatically handle ghost values, and does so in the most effficient way.

To tame the complexity of PETSc's C-style API, I utilized a **Facade Pattern**. The `MGSolver` class wraps the verbose configuration of the `KSP` and `PC` contexts, exposing only high-level parameters by means of a helper struct `KSP_params` and `MG_settings`.

4.2.2 High-Order Boundary Treatment

A critical requirement for the Multigrid solver is maintaining global second-order accuracy.

To address this, I employ a **Ghost Point Extrapolation** technique using a 3rd-order polynomial fit. By constructing a polynomial $I(x) = ax^3 + bx^2 + cx + d$ constrained by the internal pressure values ϕ_1, ϕ_2, ϕ_3 and the physical boundary condition g (where $I'(0) = g$ for Neumann or $I(0) = g_{dir}$ for Dirichlet).

$$\begin{cases} I'(0) = g \\ I(h) = \phi_1 \\ I(2h) = \phi_2 \\ I(3h) = \phi_3 \end{cases}$$

Solving this system allows to express the ghost node value (the constant term d) as a linear combination of the internal nodes and the boundary value. This relation is directly embedded into the system matrix $\mathbf{A}$, preserving the block-structured nature of the linear system while enforcing high-order consistency.

These coefficents are stored in a simple struct, that is returned by a consteval templated function to avoid useless computation at runtime and gain code clarity and modularity.

This technique is also widely used in immersed boundary methods solver, and a similar implementation can be used also for the imposition of the BC on the staggered points for the velocity tensor.

Solver setup PETSc gives an almost infinite number of possible setups, so to preserve the flexibility provided by the library there is a simple struct that helps the end user to make a setup in order to avoid to remember or make having to set an infinite amount of flags at runtime.

The final setup I chose as the default one is the following since resulted as the most stable for the tests I made of course the results may vary if the problem size increases notably.

```
struct KSP_parameters
{
    PetscScalar reltol{1e-10};
    PetscScalar abstol{1e-10};
    PetscScalar diverg_tol{1e+3};
    PetscInt    maxits{1000};
    PCType      pc_type{PCMG};
    KSPType     ksp_type{KSPGMRES};
```

```
    bool        mg_solver{true};
}

struct MG_settings
{
    int mg_levels = 4;
    int smoothing_iters = 2;
    PCMGType pc_mg_type = PC_MG_FULL;
    PCMGCycleType cycle_type = PC_MG_CYCLE_V;
    KSPType smoother_type = KSPCHEBYSHEV;
    PCType smoother_precond = PCJACOBI;
    KSPType coarse_ksp = KSPPREONLY;
    PCType coarse_pc = PCREDUNDANT;
};
```

Numerical requirements for the solver: The solver to work needs that in each direction the number of elements can be expressed as $N = C2^l + 3$, where $C, l \in Z$.

If this condition is not met the solver will fail and won't run using the setup implemented, there is no safety check to prevent the user from doing this the solver simply will skip the solve call, there will be an error dump from PETSc.

To avoid having to manually change the main file every time is possible to let the solver itself decide the numeber of levels for the multigrid hierarchy, the value is extimated if the `enforce_levels` member of the `MG_setting` is set to false (default behaviour).

The numeber of levels is extimated by taking the number of minum elements among the three directions and then computing `size_t opt = std::floor(std::log2(n_min) - 2);`.

5 Performance Analysis of the Pressure Solvers

Now that the mathematical formulation and implementation details of the two solvers have been described is time to proceed and analyze their computational performance.

The performance analysis focuses on two distinct aspects:

- **Macroscopic Scaling:** Measuring the Time-to-Solution as a function of grid size (N) and core count (P) to verify algorithmic complexity ($\mathcal{O}(N)$ vs $\mathcal{O}(N \log N)$).
- **Micro-Architectural Behavior:** Analyzing CPU counters to understand hardware bottlenecks (e.g., cache misses, branch mispredictions).

While standard timers like `MPI_Wtime` or `std::chrono` are sufficient for macroscopic measurements, they offer no insight into *why* a solver performs a certain way. To bridge this gap **Linux perf** was employed.

Unlike micro-benchmarking libraries (e.g., Google Benchmark) which can be difficult to integrate into distributed MPI environments, `perf` interacts directly with the Linux kernel's hardware performance counters. This allows for a non-intrusive measurement of instruction throughput (IPC) and memory bandwidth utilization, providing concrete evidence.

To quantify macroscopic performance, a custom ChronoTimer wrapper class was implemented to standardize timing outputs across different ranks.

5.1 Convergence

To verify the numerical efficiency and accuracy, a series of 10 cluster runs was performed (in order to minimize statistical errors).

The setup consists of 6 MPI processes scattered on 2 CPUs. The results are visible in Figure 6.

The test employs the Method of Manufactured Solutions (MMS), imposing an analytical pressure field $p_{ex} = \cos(x)\cos(y)\cos(z)$ for $\mathbf{x} \in [0, \pi]^3$. To verify the pressure solver structure, the velocity field is constructed such that the divergence constraint satisfies the Poisson equation:

$$\nabla^2 p = \nabla \cdot u \implies \nabla p = u$$

(leveraging the identity $\nabla^2 = \nabla \cdot \nabla$).

The solver initializes the velocity tensor with this manufactured field, and the resulting numerical pressure is compared against the analytical solution.

(See `tests/parallel/p_corr_basic.cpp` for implementation details).

The observed second order convergence is consistent with the theoretical expectations, confirming the correctness of the spatial discretization.

Figure 6 (right) illustrates the relationship between time-to-solution and problem size (Degrees of Freedom). Both solvers adhere closely to their theoretical complexity lines. The Fourier-based solver exhibits a slightly lower slope constant, though a larger computational campaign would be required to rigorously quantify the asymptotic scaling limits.

The logarithmic scale visually compresses the performance gap; however, Table 1 reveals a two-order-of-magnitude difference in execution time. The Multigrid solver is significantly slower than the FFT-based approach, whereas the spectral method maintains higher accuracy, as demonstrated in Figure 6. While one might attribute the spectral

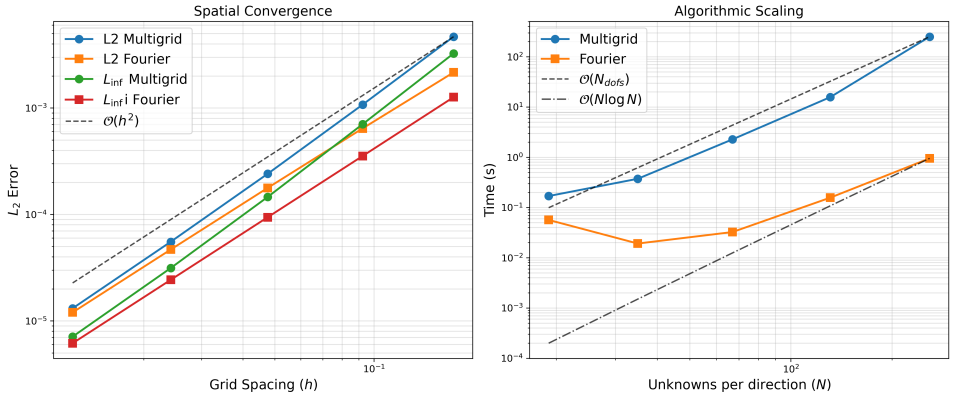

Figure 6: Spatial convergence and algorithmic scaling comparison between Multigrid and FFT solvers.

solver's exceptional precision to the alignment between the trigonometric manufactured solution and the Fourier basis, it is important to note that this superior accuracy was also observed in tests utilizing polynomial forcing terms, confirming the robustness of the method.

N	MultiGrid [s]	FFT [s]	Speedup (MG/FFT)
11	1.6364e-02	9.0618e-03	1.805
19	1.2891e-01	8.9922e-03	14.33
35	1.1482e+00	2.1489e-02	53.43
67	8.8444e+00	1.0291e-01	85.94
131	1.6261e+01	1.4968e-01	108.6
259	6.4888e+02	2.2742e+00	285.3

Table 1: Performance comparison between Geometric Multigrid and FFT-based solvers.

The inflection in the first part is due to the small number of elements; in fact, since the domain is small, most of the time is spent in the communications and transposition of data. On the Multigrid side the overhead is mitigated because the MG solver uses just 2 levels for the smoothing, and the coarser level is solved directly on rank 0 to limit the number of communications.

However, as the number of elements increases, the speedup of the spectral solver gets bigger and bigger.

5.2 Micro benchmarking with GNU/perf profiler

Methodology: Perf vs. Gprof

While macroscopic scaling tests were performed on the cluster to capture MPI interconnect behavior, micro-architectural profiling was conducted on a local workstation to bypass cluster kernel security restrictions (`perf_event_paranoid`). Since the instruction mix of the Multigrid algorithm is deterministic, the identification of bottlenecks remains valid across architectures.

Local workstation hardware specifications can be found Listing 3. `Perf` was prioritized over `gprof` because `gprof` relies on code instrumentation, in fact it modifies the binary

introducing overhead and reduces timing precision. In contrast, `perf` queries the Operating System directly with negligible overhead and is capable of providing high-fidelity hardware counter data.

In order to track performance precisely and investigate the discrepancy despite the theoretical algorithmic scaling, I chose the `perf record` and `stat` tools.

The tests were conducted on `tests/parallel/convergence_pressurecorrect.cpp`; this particular test runs the same problem using the two different solvers. After compiling using the `OPT_FLAGS` (i.e., `-O3` and `--march=native -DNDEBUG`), the test was run with `perf record -g ./build/parallel/convergence_pressurecorrect`.

As mentioned, this specific profiling run mirrors the workload of the cluster environment.

After running `perf report`, I was able to analyze the hot spots of the simulation run. The results were in line with what the macroscopic measurements revealed.

There are two important metrics to look for in this case: **Self** and **Children** percentages. These indicate respectively what percentage of the total time is employed in the function call itself and the time that *children* functions spent relatively to the parent call.

Initial inspection reveals how for a relatively small domain (the test was run on 67^3 elements) the solver spends 87% of the execution time in the multigrid solver loop while the spectral solver needed just an astonishing 0.24% of the total time of the program execution.

However, both those functions act simply as a director that calls the different functions from the needed backend; the hottest spot in the code is seen in the listing below. It is evident that `MatMult`, despite being highly optimized, still limits the solver. The dominance of `MatMult` (66% of runtime) confirms that the runtime is dictated by the iterative solver (GMRES in my settings) and the relaxation steps on the fine grids. Deactivating the multigrid option results in significantly worse solve times, also the algebraic multigrid option is far from the efficiency of the multigrid solver.

This analysis leads to the conclusion that even if the solver per se could be improved the main bottleneck is in the algorithm itself, not in the `MultiGridPoissonSolver` class.

Listing 2: CPU cycle breakdown for the Multigrid Solver obtained via Linux perf.

```
# Samples: 36K of event 'cpu_core/cycles/P'
# Event count (approx.): 20345076696
# Overhead  Command          Shared Object        Symbol
# ........  ...............  ...................  ................................
    66.11%  convergence_pre  libpetsc.so.3.24.2   [.] MatMult_SeqAIJ
     7.56%  convergence_pre  libpetsc.so.3.24.2   [.] MatPtAPSymbolic_SeqAIJ...
     5.21%  convergence_pre  libpetsc.so.3.24.2   [.] MatPtAPNumeric_SeqAIJ...
     3.31%  convergence_pre  libpetsc.so.3.24.2   [.] MatSetValues_SeqAIJ
     2.88%  convergence_pre  libpetsc.so.3.24.2   [.] DMCreateMatrix_DA_3d_MPIAIJ
     1.35%  convergence_pre  libpetsc.so.3.24.2   [.] VecSum
     0.97%  convergence_pre  [kernel.kallsyms]    [k] clear_page_erms
     0.89%  convergence_pre  libpetsc.so.3.24.2   [.] MatMultTransposeAdd_SeqAIJ
     0.87%  convergence_pre  libpetsc.so.3.24.2   [.] ISLocalToGlobalMappingApply
     0.85%  convergence_pre  libpetsc.so.3.24.2   [.] PetscSortInt
```

Why the Multigrid suffers against the spectral solver

Another issue arises due to memory-bandwidth limitation memory hierarchy pressure

.

To understand the hardware-level behavior of the Multigrid solver, I conducted a micro-architectural analysis using the `perf stat` tool. This analysis focused on identifying whether the performance is limited by computational throughput (compute-bound) or data movement (memory-bound).

Kernel Function	CPU %	Algorithmic Interpretation
`MatMult_SeqAIJ`	66.1%	Sparse Matrix-Vector Product (The Smoother)
`MatPtAP...`	12.8%	Galerkin Coarse Grid Operator Construction (RAP)
`MatSetValues...`	3.3%	Matrix Assembly / Filling values
`DMCreateMatrix...`	2.9%	Memory Allocation for Multigrid Levels
`VecSum`	1.4%	Vector Operations (likely Residual Norm calculation)

Table 2: Functional breakdown of the solver execution.

I compared two grid resolutions, $N = 67^3$ and $N = 131^3$, to observe how hardware counters scale with problem size. The scaling results are summarized in Table 3.

Metric	Grid 67^3	Grid 131^3	Scaling Factor
Time [s]	7.46	50.68	6.8
Instructions (10^9)	45.02	343.4	7.6
L1-dCache Misses (10^6)	105.9	817.7	7.7
LLC Load Misses (10^6)	7.02	59.64	**8.5**
IPC (Instr. Per Cycle)	3.20	3.12	≈ 0.98

Table 3: Hardware counter scaling between coarse and fine grids. The Last-Level Cache (LLC) misses scale faster than the instruction count, indicating increasing pressure on the memory subsystem.

The data reveals two critical insights regarding the solver's performance profile:

1. **High Instruction Throughput (IPC):** The solver maintains a high Instructions Per Cycle (IPC) count of $\approx 3.1 - 3.2$ across both grid sizes. This indicates that the CPU core is efficiently utilized and that the vectorization (AVX/AVX2/SIMD) of the stencil operations is effective. The solver is not stalling significantly on branch mispredictions or pipeline hazards due to the efficient data handling and consistent and predictable access pattern.

2. **Memory Hierarchy Pressure:** While the computational work (Instructions) scales linearly with the problem volume ($\approx 7.6\times$), the Last-Level Cache (LLC) misses grow at a faster rate ($8.5\times$).

   For the $N = 131^3$ case, the solver generates nearly **60 million LLC load misses**. This non-linear scaling of cache misses suggests that as the grid size increases, the working set begins to exceed the capacity of the L3 cache. Consequently, the latency of fetching data from main memory (DRAM) begins to act as a significant drag on performance.

While the LLC misses indicate increasing memory pressure, the high IPC (3.12) suggests the solver is not strictly stalled by bandwidth. Instead, the primary limitation is the algorithmic overhead of the sparse matrix operations, exacerbated by memory latency.

One contributing factor could be the domain decomposition strategy. While PETSc internally optimizes data handling, a pencil decomposition (row-based) might be less cache-efficient than a 3D block decomposition where data blocks are contiguous. However, given the magnitude of the cache misses, the inherent sparsity pattern remains the primary bottleneck.

5.3 Head-to-Head Micro-Architectural Comparison

To pinpoint the root cause of the performance disparity, I compared the hardware counters of both solvers for the finest grid resolution ($N = 131^3$). The results, obtained via `perf stat` on the same hardware, are presented in Table 4.

Metric	Multigrid	FFT	Ratio (MG/FFT)
Time [s]	50.68	5.44	**9.3**
Instructions (10^9)	343.42	32.99	10.4
L1-dCache Misses (10^6)	817.72	14.31	**57.1**
LLC Load Misses (10^6)	59.64	1.02	**58.5**
IPC (Instr. Per Cycle)	3.12	3.53	0.88

Table 4: Hardware counter comparison for $N = 131^3$. The Multigrid solver incurs nearly 60× more cache misses than the FFT solver, confirming a massive difference in memory efficiency.

Analysis of the Performance Gap

The profiling data highlights three critical factors contributing to the Fast Poisson Solver's superiority:

1. **Algorithmic Overhead :** The most decisive factor is the sheer volume of instructions. The Multigrid solver executes over **343 billion** instructions, whereas the FFT solver requires only **33 billion** (10× fewer) to solve the same physical problem. This 10-fold difference in work directly correlates with the 9.3x difference in runtime, indicating that the sparse matrix logic and indexing and conditionals needed for the Geometric multigrid extrapolations impose a massive constant factor overhead compared to the dense, streamlined arithmetic of the FFT.

2. **Memory Locality:** The most striking difference is in the Last-Level Cache (LLC) load misses. The Multigrid solver generates nearly **60 million** of them, implying constant traffic between the CPU and main memory (DRAM). In contrast, the FFT solver generates only **1 million** misses, same goes for the L1d (Data L1) Cache misses.

   This indicates that the `FFTW3` library effectively utilizes cache-oblivious algorithms, keeping the active working set entirely within the L3 cache. This 60x difference in cache misses acts as a latency multiplier on the already high instruction count of the Multigrid solver.

3. **Vectorization Efficiency (IPC):** The FFT solver achieves a higher Instructions Per Cycle (IPC) of 3.53 compared to 3.12 for Multigrid. This suggests that the dense linear algebra operations inside the FFT are more amenable to SIMD (AVX/AVX2) vectorization than the sparse, memory-strided loops of the Multigrid smoother.

While the Multigrid method is algorithmically scalable, its practical performance on this grid size is severely hampered by the high algorithmic overhead (instruction count) and the overhead of sparse data structures. The Spectral solver, conversely, maximizes modern hardware capabilities by minimizing data movement and computational work , resulting in an order-of-magnitude speedup.

5.4 Scaling

As visible in image Figure 7 the scaling of the 2 different methods is not ideal. The **Multigrid** solver exhibits more robust scaling characteristics due to the optimized halo-exchange implementation provided by PETSc. Conversely, the **Spectral** solver demonstrates rapid saturation, largely attributable to the `MPI_Alltoallv` global transpositions required for the pencil decomposition.

The speedup analysis in Figure 7 shows how the solvers stall and in particular how the **Spectral** one has to say the least a bad scaling, the small domain may have an impact on the scaling.

Those tests were made on the local machine (Details in Listing 3) using a cubic domain of 67 elements in each direction. Given the high speed of the FFT algorithm, the local computation time is extremely short, making the communication overhead the dominant factor. It is also worth noting that the 8 MPI processes were scheduled across a hybrid architecture (P-cores and E-cores), introducing load imbalance and overhead.

The most reasonable scaling is the one from 1 to 4 cores. Furthermore, since this test utilized a shared-memory architecture (single node), communication latency was minimized compared to a distributed cluster environment.

A distributed memory model has advantages when scaling to hundreds of cores, but the communications take much more time and their overhead could be relevant if the solver domain is too small.

The conclusion is that this scaling analysis is not of generic relevance and may have quantitatively different outcomes based on the architecture of the host machine, but the qualitative trends should confirm that the Multigrid benefits from local neighbor-only communications, scaling better than the Spectral solver which relies on global all-to-all transpositions.

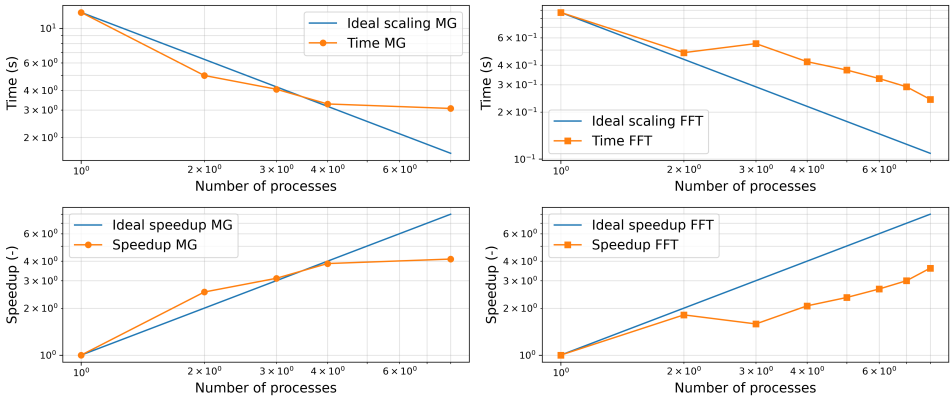

Figure 7: Scaling with different number of MPI processes.

6 Full solver implementation

A specific validation test was developed based on the Chorin-Temam projection operator, based on the Helmotz decomposition, to ensure the divergence free condition is imposed correctly. The velocity field is initialized as a perturbed solenoidal field: $\mathbf{u} = \mathbf{u}_{\text{div-free}} + \nabla\phi$. The pressure correction step is then applied, and the resulting velocity field is checked against the analytical divergence-free solution.

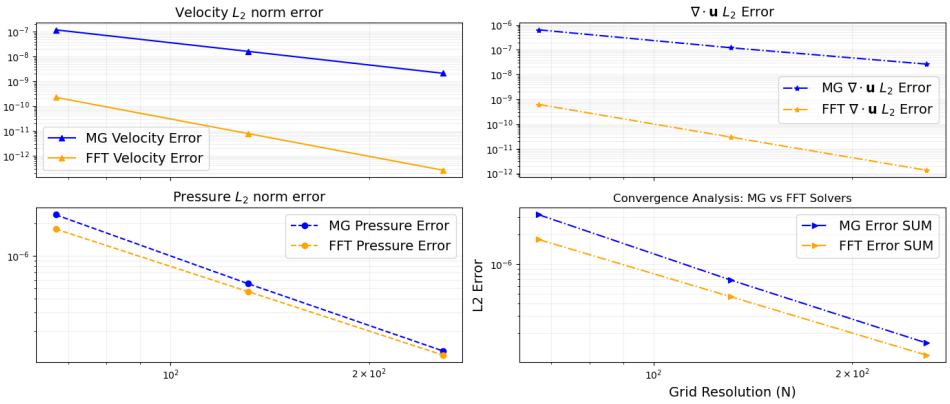

Figure 8: Convergence analysis of the pressure correction step. The plots compare L_2 errors for pressure, velocity, and final divergence.

The results, visualized in Figure 8 and detailed in Table 5, reveal a distinct trade-off between the two methods:

- **Velocity recovery:** Both solver result in correct velocity correction, again, the FFT based solver has the edge.
- **Pressure Accuracy:** The Multigrid solver achieves L_2 errors for the pressure scalar field that are very close to the one of the Spectral solver.
- **Divergence-free imposition:** The Spectral solver is significantly more effective at enforcing the physical constraint $\nabla \cdot \mathbf{u} = 0$. As shown in Table 5, the spectral method reduces divergence to a factor of 2 against the MG solver.

Solver	N	Velocity Recovery	Pressure Error	Final Divergence
MG	67	1.19-07	2.40-06	6.38-07
	131	1.62-08	5.53-07	1.20-07
	259	2.11-09	1.31-07	2.63-08
FFT	67	2.26-10	1.77-06	6.14-10
	131	7.84-12	4.68-07	2.93-11
	259	2.59-13	1.20-07	1.35-12

Table 5: Comparison of L_2 Error Norms: MultiGrid (MG) vs. FFT Solvers.

Table 6 presents how both solvers empirical orders of convergence. Both solvers exhibit "super-convergence" for the velocity field.

Order ≈ 5 for the Spectral solver one may point to the smoothness of the manufactured solution being perfectly captured by the Fourier basis, however is worth notice that the

spectral solver has shown to be quite robust, and only very hard discontinuities make it perform bad, but this is not the case.

The Multigrid solver performs well, but still shows to be less accurate one may argue that this is due to the tolerances, to assess this hypothesis one may just pay the price of having longer runtime.

Solver	Pressure	Velocity	$\nabla \cdot \mathbf{u}$
Multigrid	2.15	2.98	2.36
Spectral	1.99	5.01	4.53

Table 6: Observed order of convergence for various physical quantities.

To conclude the comparative analysis, I evaluated the solvers in a realistic simulation context using the `NSSolver` class. This is the moment where problems arose since the single solver steps are working, however, when integrated the complete solve fails leading to unexpected numerical errors and in some cases inverse convergence over time, the issues could simply be related to the nature of the problem formulation itself, in fact Navier-Stokes equation are highly sensitive to difference in initial condition, this means that there may be points where the solver diverges simply due to the nature of the physical problem despite the implementation.

To address this doubt one could run the simulations on a super fine grid so that the spacing matches the already mentioned Kolmogorov's scale at a prohibitive cost and therefore resolve the turbulence at its finest levels (as the solver expects to work being a DNS solver) and provide convergence tests. Given the lack of computational power I cannot resolve the doubt and I must limit my analysis to small test cases in laminar regime, is the case of the main test.

Here the pressure solver simply expects a velocity field to solve for, in the provided test case the exact solution is a simple velocity field with $\mathbf{U}(x, y, z) = [y, 0, 0]^T$, this trivial test works, however it cannot be used to assess the goodness of the solver, can however be seen as a sanity check.

The test I ran with velocity field different from the mentioned one resulted in chaotic convergences, the general trend is that in each time step the divergence of the fluid field increases and the pressure correction step is able to lower it, however the simulations keep diverging leading to non-consistent and non-publishable results.

I still included the NSSolver architecture even tho is not fully working both for the heavy work it required to implement it and to show how different classes can be integrated in the same workflow by means of concepts, templated policies and in general avoiding virtual classes to promote a different approach from the usual.

7 Conclusions

In conclusion, the spectral solver demonstrates superior performance for this application. It is computationally faster and provides a more precise projection for the velocity field. While Multigrid offers geometric flexibility and scales more effectively on the tested configurations, the performance penalty makes the Spectral approach the definitive choice for simple domain DNS.

Is worth notice that the spectral approach may be applied to non-simple parallelepipedic domains by means of Schur complement however this extension is way over the purpose of this project.

As final note, I wanted to leave a *reflection* (Despite not being in `C++`26 yet) on what could have been done differently.

Mainly the first possible difference is that the pressure mesh could have been positioned half step staggered in all three directions, this would have allowed to avoid the extrapolate_div functions since the divergence on the faces would not be needed, however due to inexperience and lack of foresight I did not do this.

Another difference could be about the mesh dimensions, in fact one could argue (with a fair point) that the values on the sides are not needed and could have been enforced directly into the inner layers when imposing the boundary conditions, this however would make the handling of different BCs quite a hard job, in fact in case of periodic BC there would be the need for more communications, leading to confusion and possible culprit falling in the case of pre-premature optimization.

A factory pattern could have been implemented as well, however I preferred to keep the main a bit more verbose and self descriptive, given that a factory class would have then took CLI arguments directly hiding too much of the actual implementation in my view, the 30 lines in the main allow to keep a more explicit way of setting and handling the implementation, on the other hand a factory could help to maintain a more controlled initialization and general control over the full solver implementation providing a more robust approach.

Also I want to point out how the big $\mathcal{O}$ notation can be sometimes deceiving and most importantly how the solver algorithm can impact the runtime much more than the implementation, I in fact doubt that my implementation of the Fast Poisson solver is state of the art, and I am instead sure that if I would have implemented the Multigrid Solver I would have made it much less efficient than the state of the art implementation provided by the PETSc library; but most importantly how in general is the math done on *paper* and the reasoning on the implementation that can make the difference between an efficient, faster and more precise solver.

A final note of appraisal goes to my friends and colleagues Pietro and Matteo, whom with I shared the learning process and enthusiasm of learning `C++` features and development of fast performant code.

Thank you for the attention.

Matteo

8 Extra

8.1 Cpu informations

Listing 3: Local CPU informations

```
    Architecture:             x86_64
  CPU op-mode(s):             32-bit, 64-bit
  Address sizes:              39 bits physical, 48 bits virtual
  Byte Order:                 Little Endian
CPU(s):                       12
  On-line CPU(s) list:        0-11
Vendor ID:                    GenuineIntel
  Model name:                 13th Gen Intel(R) Core(TM) i7-1355U
    Thread(s) per core:       2
    Core(s) per socket:       10
    Socket(s):                1
    Stepping:                 3
    CPU(s) scaling MHz:       22%
    CPU max MHz:              5000.0000
    CPU min MHz:              400.0000
Caches (sum of all):
  L1d:                        352 KiB (10 instances)
  L1i:                        576 KiB (10 instances)
  L2:                         6.5 MiB (4 instances)
  L3:                         12 MiB (1 instance)
```